MODÈLES GÉNÉRATIFS PROFONDS

Mini-projets pratiques

Un mini-projet pour chaque bloc de deux semaines

S1–S2 · S3–S4 · S5–S6 · S7–S8 · S9–S10 · S11–S12 · S13–S14

Objectif : mettre en pratique les concepts essentiels sans transformer les activités en gros projets.

Principes généraux

Chaque mini-projet est conçu pour être réalisable en quelques heures et sert à **mettre en pratique les concepts vus pendant les deux semaines correspondantes**. L'objectif n'est pas d'obtenir le meilleur score possible, mais de comprendre le comportement, les hypothèses et les compromis des modèles.

— Fashion-MNIST et MNIST servent de jeux de données fil rouge pour faciliter les comparaisons.

— Les jeux de données synthétiques 2D sont privilégiés lorsque la visualisation des densités, champs de vecteurs ou trajectoires est essentielle.

— Chaque mini-projet comporte exactement **5 éléments à évaluer** et **5 questions clés de réflexion**.

Livrable commun

Pour chaque mini-projet, remettre :

1. un carnet Jupyter reproductible ou un script clairement organisé ;
2. les cinq éléments demandés dans la section **5 éléments à évaluer** ;
3. des réponses courtes aux cinq questions clés ;
4. une conclusion de 150 à 250 mots reliant les résultats aux concepts du cours.

Problème à traiter

Important : l'évaluation porte autant sur l'interprétation probabiliste des résultats que sur le fonctionnement du code. Il n'est pas demandé d'optimiser les modèles jusqu'à obtenir des performances de référence.

1 Mini-projet 1 — Semaines 1–2

Représentations apprises : classifier n'est pas générer

Description

Entraîner un petit CNN discriminatif, extraire sa représentation interne et visualiser les plongements afin de comprendre la différence entre représentation, classification et génération.

Jeu de données

Fashion-MNIST : images 28×28 en niveaux de gris, 10 classes de vêtements.

Lien : https://github.com/zalandoresearch/fashion-mnist

Problème à traiter

Entraîner un petit CNN pour reconnaître les 10 catégories. Récupérer ensuite le vecteur de caractéristiques juste avant la couche de classification et projeter ces représentations en 2D avec PCA. Le but est d'observer si les classes se regroupent et d'expliquer pourquoi cette représentation n'est pas encore un modèle génératif.

Modèle / méthode à utiliser

Petit CNN :

$$28 \times 28 \to \text{Conv} \to \text{Conv} \to z \to \text{Linear}(10).$$

5 éléments à évaluer

1. Accuracy du classificateur sur le jeu de test.
2. Visualisation PCA des plongements.
3. Distance entre plongements de deux exemples d'une même classe.
4. Distance entre plongements de classes différentes.
5. Explication de la raison pour laquelle un latent arbitraire ne permet pas automatiquement de générer une image plausible.

5 questions clés

1. Où se trouve exactement la représentation apprise dans le CNN ?
2. Deux images proches dans l'espace latent sont-elles forcément similaires sémantiquement ?
3. Le CNN apprend-il $p(y \mid x)$, $p(x)$ ou $p(x, y)$?
4. Pourquoi ce réseau est-il discriminatif et non génératif ?
5. Que faudrait-il ajouter pour passer de $x \to z \to y$ à un mécanisme capable de produire un nouvel x ?

2 Mini-projet 2 — Semaines 3–4

Maximum de vraisemblance et variable latente

Description

Mettre en pratique MLE, log-vraisemblance, variables latentes et inférence à l'aide d'un exemple 2D totalement visualisable.

Jeu de données

Mélange synthétique de deux gaussiennes 2D. Aucun téléchargement n'est nécessaire.

Problème à traiter

Générer des observations selon

$$p(x) = 0.5\,\mathcal{N}(\mu_1, \Sigma_1) + 0.5\,\mathcal{N}(\mu_2, \Sigma_2),$$

par exemple avec $\mu_1 = (-2, 0)$ et $\mu_2 = (2, 0)$. Ajuster d'abord une gaussienne unique, puis un mélange de deux gaussiennes et interpréter la loi a posteriori $p(z \mid x)$.

Modèle / méthode à utiliser

Utiliser un **modèle de mélange gaussien**, par exemple `sklearn.mixture.GaussianMixture`. La variable latente $z \in \{0, 1\}$ représente la composante ayant généré chaque point.

5 éléments à évaluer

1. Estimation correcte des paramètres de la gaussienne ou des composantes.
2. Calcul ou affichage de la log-vraisemblance.
3. Comparaison gaussienne unique ou mélange de deux gaussiennes.
4. Visualisation des deux composantes latentes.
5. Calcul ou interprétation de la loi a posteriori $p(z \mid x)$.

5 questions clés

1. Pourquoi maximise-t-on la log-vraisemblance plutôt que la vraisemblance brute ?
2. Que représente ici la variable latente z ?
3. Pourquoi z est-elle dite non observée ?
4. Quelle différence existe entre apprendre les paramètres du modèle et inférer z ?
5. Pourquoi le calcul de $p(z \mid x)$ devient-il plus difficile avec un modèle latent profond ?

3 Mini-projet 3 — Semaines 5–6

Génération auto-régressive ou reconstruction par autoencodeur

Description

Comparer la génération composante par composante d'un modèle auto-régressif à la reconstruction globale d'un autoencodeur.

Jeu de données

MNIST : images 28×28 de chiffres manuscrits. Pour réduire le coût, utiliser environ 10 000 images ou une résolution 14×14.

Lien : https://yann.lecun.com/exdb/mnist/

Problème à traiter

Construire deux petits modèles : (A) un modèle auto-régressif sur MNIST binarisé,

$$p(x) = \prod_i p(x_i \mid x_{<i}),$$

et (B) un autoencodeur

$$x \to z \to \hat{x}.$$

Comparer surtout leurs mécanismes et non leur performance absolue.

Modèle / méthode à utiliser

Un MADE ou réseau masqué simple, et un autoencodeur dense ou convolutionnel léger.

5 éléments à évaluer

1. NLL du modèle auto-régressif.
2. Temps de génération de plusieurs images auto-régressives.
3. Erreur de reconstruction de l'autoencodeur.
4. Visualisation de l'espace latent de l'autoencodeur.
5. Test de génération en décodant des z aléatoires.

5 questions clés

1. Pourquoi le modèle auto-régressif possède-t-il une vraisemblance calculable ?
2. Pourquoi sa génération est-elle séquentielle ?
3. Pourquoi l'autoencodeur peut-il reconstruire une image rapidement ?
4. Pourquoi un excellent autoencodeur peut-il produire de mauvaises images avec $z \sim \mathcal{N}(0, I)$?

5. Quelle limite de l'autoencodeur motive directement le VAE ?

4 Mini-projet 4 — Semaines 7–8

Deux façons de structurer un espace latent : VAE ou flot

Description

Comparer un VAE entraîné avec une ELBO et un flot normalisant qui impose l'inversibilité pour calculer une vraisemblance exacte.

Jeu de données

Jeu de données principal : Fashion-MNIST. Le VAE est entraîné sur les images 28×28.

Lien : https://github.com/zalandoresearch/fashion-mnist

Pour le flot, on travaille volontairement dans un **espace 2D continu et visualisable**, construit à partir de Fashion-MNIST : PCA 2D ou codes latents 2D du VAE. Ainsi, RealNVP n'est pas entraîné directement sur les 784 pixels.

Problème à traiter

Construire un VAE sur Fashion-MNIST avec un latent

$$z \in \mathbb{R}^2.$$

Pour le flot, ne pas entraîner RealNVP directement sur les images 28×28 : utiliser soit :

— les deux premières composantes d'une PCA appliquée aux images ;

— les codes latents 2D extraits par le VAE après entraînement ;

— un sous-ensemble limité à deux classes, puis projeté en 2D.

Observer la différence entre latent régularisé, ELBO, inversibilité et vraisemblance exacte dans cet espace 2D visualisable.

Modèle / méthode à utiliser

VAE dense ou convolutionnel léger ; RealNVP 2D avec quelques couches de couplage. Le flot doit rester volontairement petit : deux à quatre couches de couplage suffisent pour cette activité.

5 éléments à évaluer

1. Qualité de reconstruction du VAE.
2. Organisation du latent z du VAE.
3. Échantillons générés avec $z \sim \mathcal{N}(0, I)$.
4. Vérification numérique de $f^{-1}(f(z)) \approx z$.
5. Log-vraisemblance calculée par le flot.

5 questions clés

1. Quel est le rôle du terme reconstruction dans l'ELBO ?
2. Quel est le rôle de $D_{\mathrm{KL}}(q_\phi(z \mid x) \| p(z))$?
3. Pourquoi le VAE permet-il d'échantillonner plus naturellement qu'un AE classique ?
4. Pourquoi RealNVP impose-t-il des transformations inversibles ?

5. Quelle différence essentielle existe entre ELBO et vraisemblance exacte ?

5 Mini-projet 5 — Semaines 9–10

Énergie ou génération implicite

Description

Comparer un EBM, qui apprend une fonction d'énergie, et un GAN, qui apprend directement une procédure de génération implicite.

Jeu de données

Jeu de données : mélange de 8 gaussiennes 2D.
Les centres sont placés régulièrement sur un cercle, par exemple

$$\mu_k = r \begin{pmatrix} \cos(2\pi k/8) \\ \sin(2\pi k/8) \end{pmatrix}, \qquad k = 0, \dots, 7.$$

Chaque observation est tirée autour d'un centre avec une petite covariance isotrope. Aucun téléchargement n'est nécessaire : le jeu de données est généré directement avec NumPy.

Ce choix est préférable à Two Moons pour ce bloc, car les **8 modes distincts** permettent de voir immédiatement si un GAN couvre toute la distribution ou s'il ignore certains modes.

Problème à traiter

Apprendre (A) une fonction d'énergie $E_\theta(x)$ et visualiser son paysage, puis (B) un petit GAN 2D

$$z \to G_\theta(z) \to x.$$

Comparer les régions d'énergie faible, les échantillons Langevin et les échantillons GAN.

Modèle / méthode à utiliser

Petit MLP comme fonction d'énergie. Utiliser des chaînes de Langevin courtes pour produire les échantillons négatifs ; une banque persistante de particules est recommandée mais optionnelle. Petit GAN composé de deux MLP, entraîné sur le même jeu de données. L'objectif reste qualitatif : visualiser le paysage d'énergie, la dynamique des échantillons et la couverture des modes.

5 éléments à évaluer

1. Carte du paysage d'énergie.
2. Position des vraies données dans les régions d'énergie faible.
3. Échantillons générés par Langevin.
4. Échantillons produits par le GAN.
5. Nombre de modes couverts parmi les 8 gaussiennes et répartition approximative des échantillons entre ces modes.

5 questions clés

1. Pourquoi une faible énergie correspond-elle à une configuration plausible ?
2. À quoi sert la fonction de partition $Z(\theta)$?
3. Pourquoi l'apprentissage d'un EBM nécessite-t-il de l'échantillonnage, et pourquoi des chaînes

de Langevin courtes ne donnent-elles que des échantillons approximatifs ?

4. Pourquoi un GAN peut-il générer sans calculer $p_\theta(x)$?
5. Comment détecter un effondrement des modes lorsque la distribution réelle possède 8 modes ?

6 Mini-projet 6 — Semaines 11–12

Du score au modèle de diffusion

Description

Construire un mini-DDPM afin de rendre concret le lien entre score, bruitage progressif, prédiction du bruit et génération inverse.

Jeu de données

MNIST. Utiliser un sous-ensemble d'environ 10 000 images pour garder l'entraînement léger.

Lien : https://yann.lecun.com/exdb/mnist/

Problème à traiter

Visualiser le processus direct

$$x_0 \to x_t \to x_T.$$

Pendant l'entraînement, obtenir directement une version bruitée de x_0 :

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Entraîner ensuite un réseau à prédire

$$\epsilon_\theta(x_t, t) \approx \epsilon.$$

Enfin, générer des images en partant de

$$x_T \sim \mathcal{N}(0, I)$$

et en appliquant le processus inverse appris.

Modèle / méthode à utiliser

Petit U-Net ou réseau convolutionnel léger conditionné par un plongement du temps t.
Pour conserver une activité de quelques heures, utiliser un sous-ensemble du jeu de données, un nombre réduit d'étapes de diffusion, par exemple $T \in [50, 100]$, et un réseau volontairement léger.

5 éléments à évaluer

1. Visualisation du processus direct pour plusieurs valeurs de t.
2. Courbe de la perte MSE sur le bruit.
3. Qualité du débruitage à différents niveaux de bruit.
4. Échantillons générés depuis $x_T \sim \mathcal{N}(0, I)$.
5. Comparaison qualitative d'un calendrier de bruit linéaire et d'un calendrier de bruit cosinus.

5 questions clés

1. Que représente t pour le réseau ?
2. Pourquoi peut-on générer directement x_t à partir de x_0 pendant l'entraînement ?
3. Pourquoi le réseau prédit-il souvent ϵ plutôt que directement x_0 ?
4. Quel lien existe entre le débruitage et le score ?
5. Pourquoi l'échantillonnage DDPM est-il beaucoup plus lent qu'un GAN ?

7 Mini-projet 7 — Semaines 13–14

Génération dans un espace latent et appariement de flot

Description

Combiner compression latente et transport continu pour illustrer l'idée des modèles hybrides modernes sans entraîner un grand modèle.

Jeu de données

Fashion-MNIST.

Lien : https://github.com/zalandoresearch/fashion-mnist

Problème à traiter

Entraîner d'abord un petit autoencodeur :

$$x \to z.$$

Encoder ensuite les images du jeu de données afin d'obtenir des codes

$$z_1 \sim p_{\text{latent}}.$$

Pour chaque code z_1, tirer indépendamment

$$z_0 \sim \mathcal{N}(0, I),$$

puis choisir

$$t \sim \mathcal{U}(0, 1).$$

Construire le point intermédiaire et sa vitesse cible :

$$z_t = (1-t)z_0 + tz_1, \qquad u_t = z_1 - z_0.$$

Le réseau apprend alors

$$v_\theta(z_t, t) \approx u_t.$$

Après intégration d'une ODE depuis un bruit initial, décoder le latent généré.

Modèle / méthode à utiliser

Petit autoencodeur + MLP pour $v_\theta(z_t, t)$.

Échantillonnage : intégrer l'ODE

$$\frac{dz_t}{dt} = v_\theta(z_t, t), \qquad z(0) \sim \mathcal{N}(0, I),$$

avec Euler explicite :

$$z_{t+\Delta t} = z_t + \Delta t\, v_\theta(z_t, t).$$

Pour garder l'activité légère, utiliser un latent de dimension 2 ou 8 et environ 20 à 50 pas d'Euler. L'autoencodeur peut être fourni pré-entraîné si le temps de séance est limité.

5 éléments à évaluer

1. Erreur de reconstruction de l'autoencodeur.
2. Dimension et visualisation de l'espace latent.
3. Perte d'appariement de flot $\|v_\theta(z_t, t) - u_t\|^2$.
4. Visualisation des trajectoires de quelques points latents.
5. Images obtenues après décodage des latents générés.

5 questions clés

1. Pourquoi est-il avantageux de générer dans l'espace latent plutôt que directement dans les pixels ?
2. Que représentent z_0, z_1 et z_t ?
3. Quelle quantité le réseau d'appariement de flot doit-il prédire ?
4. Quelle différence fondamentale existe entre l'appariement de flot et DDPM ?
5. En quoi ce mini-projet illustre-t-il une hybridation de plusieurs idées vues pendant le cours ?

Charge de travail recommandée

Projet	Risque	Garde-fou recommandé
1	Faible	3 à 5 époques de CNN suffisent.
2	Très faible	Utiliser `GaussianMixture` et imposer les visualisations.
3	Élevé	MADE simplifié, images 14×14, 5 000 à 10 000 exemples.
4	Moyen	VAE 2D + RealNVP uniquement dans un espace 2D.
5	Élevé	MLP 2D, nombre fixe de pas Langevin, GAN minimal.
6	Très élevé	Sous-ensemble de MNIST, $T \in [50, 100]$, U-Net très léger.
7	Moyen	AE pré-entraîné possible, latent 2 ou 8 dimensions, Euler 20 à 50 pas.

Vue d'ensemble

Semaines	Mini-projet	Jeu de données	Concept principal
S1–S2	Représentations et discriminatif/génératif	Fashion-MNIST	plongement, $p(y \mid x)$ ou $p(x)$
S3–S4	MLE et latent caché	Mélange gaussien 2D	vraisemblance, loi a posteriori
S5–S6	Auto-régressif ou Autoencodeur	MNIST	factorisation ou reconstruction
S7–S8	VAE ou flot	Fashion-MNIST	ELBO ou vraisemblance exacte
S9–S10	EBM ou GAN	8 gaussiennes 2D	énergie ou génération implicite
S11–S12	Mini-DDPM	MNIST	score, bruit et diffusion
S13–S14	Latent et appariement de flot	Fashion-MNIST	hybridation moderne

Principe pédagogique final : utiliser Fashion-MNIST et MNIST comme jeux de données fil rouge, et réserver les petits jeux de données 2D aux modèles pour lesquels la visualisation géométrique est essentielle.